COLLEGE MINI PROJECT PRESENTATION

Break-Even Point Predictor & Feasibility Engine

PROJECT METADATA

Domain: FinTech / Financial Engineering & Web Systems

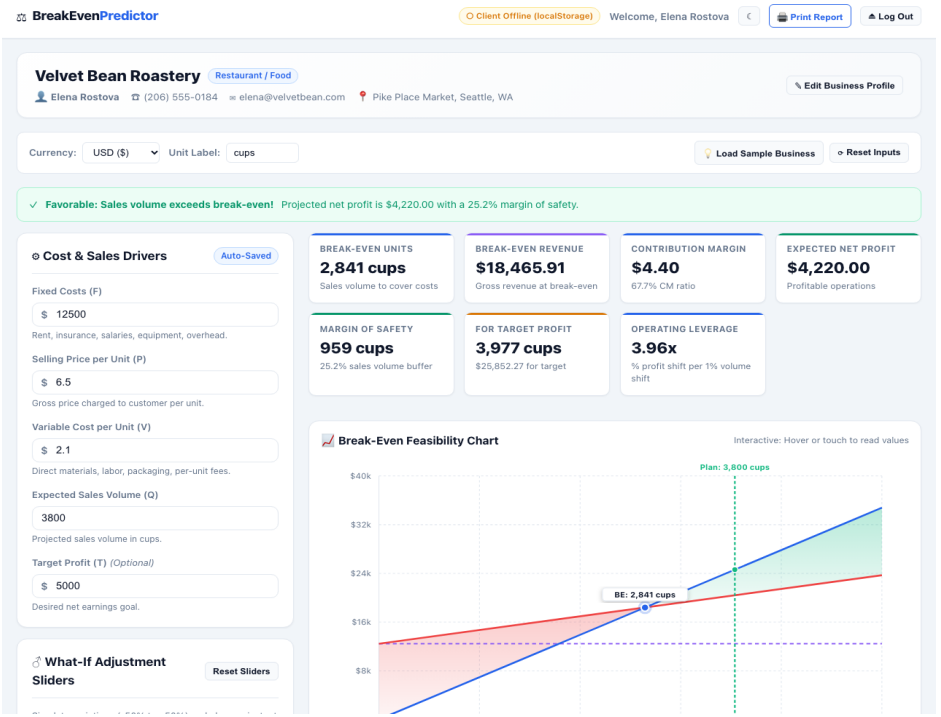
Developer: arjun2007arush2011-max

Architecture: Zero-Dependency Vanilla ES5 + Multi-Runtime REST

Database: SQLite3 Relational DB + LocalStorage Fallback

Live URL: https://arjun2007arush2011-max.github.io/break_even_point/

GitHub: https://github.com/arjun2007arush2011-max/break_even_point



BACKGROUND & MOTIVATION

Problem Statement & Project Objectives

THE PROBLEM STATEMENT

• **High Failure Rate:** Over 65% of startup failures stem from premature cost miscalculations and hidden overhead.

• **Static Spreadsheet Errors:** Traditional Excel files lack interactive dynamic curves and mobile responsiveness.

• **Blind Pricing Decisions:** Entrepreneurs often underprice without analyzing variable cost compression on margins.

• **No Sensitivity Simulation:** Business owners struggle to forecast the impact of 15% cost inflation or rent spikes.

KEY PROJECT OBJECTIVES

✔ **Zero-Dependency Web Platform:** Completely self-contained vanilla ES5 implementation running without CDNs or frameworks.

✔ **Dynamic SVG Graphics:** Visual curves for Total Revenue, Total Cost, Fixed Costs, and profit/loss zones.

✔ **Real-Time What-If Engine:** Sliders (-50% to +50%) and sensitivity tables to test cost and price variations.

✔ **Multi-Runtime REST Integration:** Idiomatic backends in Python, Ruby, and Node.js with SQLite3 persistence.

✔ **Executive Feasibility Reports:** Formatted printable reports for stakeholders with company letterhead.

FRONTEND TIER

Frontend Architecture & User Interface Design

UI DESIGN & ARCHITECTURE

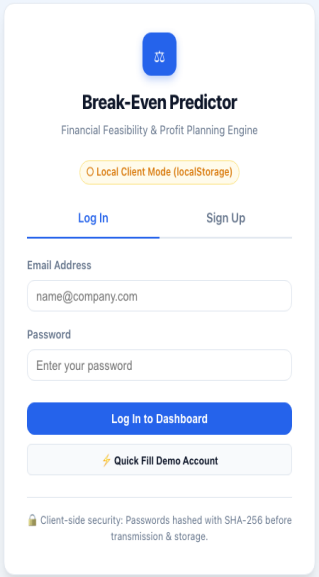
• **Vanilla ES5 Standard:** Ensures 100% universal browser compatibility without build pipelines.

• **Responsive Two-Column Layout:** Smoothly adapts between desktop and mobile screens.

• **Dark / Light Themes:** Powered by CSS Custom Properties with persistent storage.

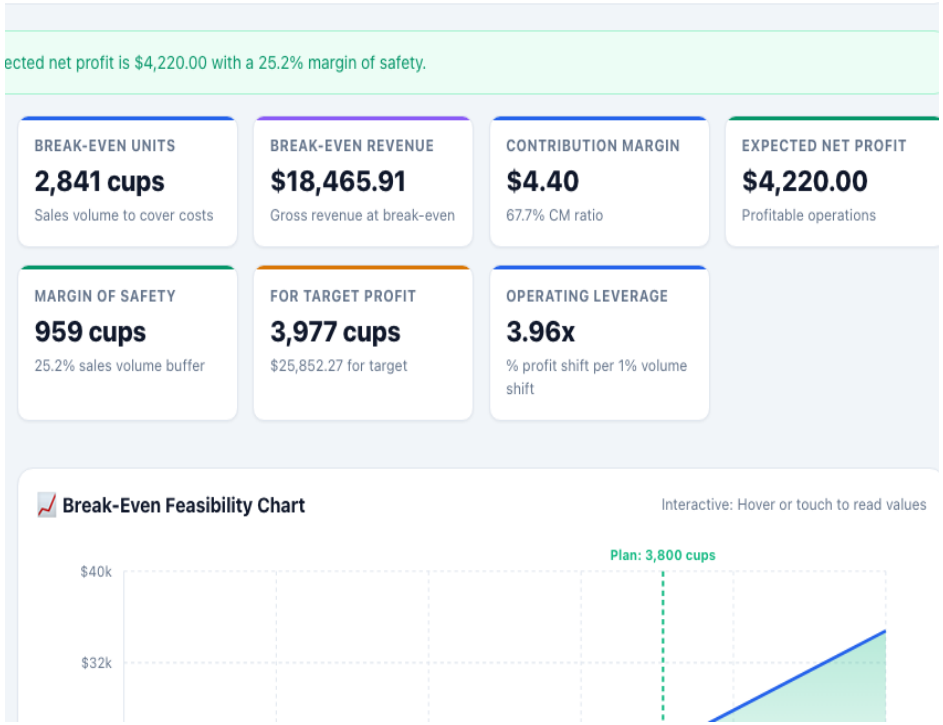
• **Client-Side SHA-256 Hashing:** Passwords hashed via crypto.subtle before storage or transmission.

• **Clean Modular Objects:** AuthModule, ProfileModule, MetricsModule, ChartModule, and ApiClient.



VISUALIZATION ENGINE

Interactive SVG Chart & Visual Capabilities



CUSTOM COORDINATE MAPPING ENGINE

- **Zero External Chart Libraries:** Custom SVG mathematical projection avoiding heavy Chart.js or D3.js libraries.
- **Loss & Profit Polygons:** Fills red loss zone (0 to BE) and green profit zone (BE to Max Volume) dynamically.
- **Break-Even Point Marker:** Displays intersection point with a pulsing halo and coordinate callout badge.
- **Expected Plan Line:** Vertical dashed indicator showing planned sales volume versus break-even.
- **Touch & Hover Crosshairs:** Real-time cursor reading volume, revenue, costs, and profit at 60fps.

BACKEND & DATABASE

Backend Architecture, REST APIs & SQLite3 Storage

TRIPLE RUNTIME BACKEND IMPLEMENTATION

• **Python 3 Backend (server.py):** Built with native http.server and sqlite3. Zero external pip packages.

• **Ruby Backend (server.rb):** Built with WEBrick and native sqlite3 gem. Zero external dependencies.

• **Node.js Backend (server.js):** Native http and crypto standard library.

• **Auto-Launcher (start.sh):** Automatically detects available runtime and launches on port 8000.

• **SQLite Database (breakeven.db):** Tables for Users, Profiles, and Cost Inputs.

REST API CONTRACT & DUAL SYNC

• **POST /api/auth/signup & login:** SHA-256 auth returning 24-byte bearer tokens.

• **GET /api/auth/me:** Validates bearer session and returns profile & inputs.

• **GET & POST /api/profile:** SQLite3 persistence for business letterhead details.

• **GET & POST /api/inputs:** Persists cost drivers with debounced auto-saving.

• **POST /api/calculate:** Server-side model calculation verification.

• **Dual-Engine Synchronization:** Probes /api/health; if online, syncs with SQLite3; if offline, operates via localStorage seamlessly.

ANALYTICS ENGINE

Mathematical Model & What-If Simulations

CORE ACCOUNTING FORMULAS

- **Contribution Margin (CM):** CM = P - V
- **Contribution Margin Ratio:** CM% = (P - V) / P
- **Break-Even Volume:** Q_BE = F / (P - V)
- **Break-Even Revenue:** R_BE = Q_BE × P
- **Net Operating Profit:** Profit = Q × (P - V) - F
- **Margin of Safety:** MOS = (Q - Q_BE) / Q × 100%
- **Operating Leverage:** DOL = (Q × CM) / Profit
- **Target Profit Units:** Q_Target = (F + Target) / CM

WHAT-IF SLIDERS & SCENARIO MATRICES

- **Real-Time What-If Sliders:** Adjust Price, Var Cost, and Fixed Costs (-50% to +50%) with live variance indicators.
- **Sensitivity Matrix:** Displays BE Volume shifts at -20%, -10%, 0%, +10%, +20% for each cost driver.
- **Volume Scenario Analysis:** Evaluates Pessimistic (-25%), Base, and Optimistic (+25%) market conditions.
- **Structural Loss Guard:** Raises instant alert if P ≤ V (selling below variable cost) with full loss shading.

CONCLUSION & IMPACT

Project Results, Live Deployment & Academic Scope

KEY DELIVERABLES & OUTCOMES

✔ **Live Production Deployment:** Hosted globally on GitHub Pages at https://arjun2007arush2011-max.github.io/break_even_point/

✔ **Zero Dependencies:** Operates without npm packages, external build tools, or CDN dependencies.

✔ **High Performance:** Sub-millisecond mathematical calculations and smooth 60fps SVG chart recalculations.

✔ **Print Feasibility Report:** Clean @media print executive feasibility report for stakeholders.

✔ **SEO Optimization:** Integrated robots.txt, sitemap.xml, and OpenGraph social metadata.

ACADEMIC IMPACT & FUTURE SCOPE

• **Cross-Disciplinary Integration:** Merges software engineering, managerial cost accounting, database design, and interactive computer graphics.

• **Multi-Product Sales Mix:** Future expansion to calculate weighted average contribution margins for multi-item inventory catalogs.

• **Monte Carlo Risk Modeling:** Simulating demand curve fluctuations and price elasticity for probabilistic risk assessment.

• **GitHub Repository:** https://github.com/arjun2007arush2011-max/break_even_point